RAMDEOBABA UNIVERSITY, NAGPUR

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

B.Tech. CSE (Artificial Intelligence and Data Science) (AIDS)

SEMESTER V

STATISTICAL INFERENCE LAB (SI LAB)

PROJECT REPORT

REGRESSION: FOOD DELIVERY TIME PREDICTION

Submitted by

Student Name: Achal Lohiya

Roll Number: 25 (Batch B2-B2 / SI Roll No. 02)

Enrollment Number: RBU/CSE-AIDS/2024/025

Programme & Year: B.Tech. CSE (AI & DS) — 3rd Year

Academic Year: 2026–27

1. PROJECT OBJECTIVE

The primary objective of this project is to develop, evaluate, and deploy an end-to-end Machine Learning regression system capable of accurately predicting the estimated food delivery duration in minutes for online orders. Accurate delivery time estimation represents a cornerstone of customer satisfaction, fleet routing efficiency, and operational dispatching in modern on-demand delivery logistics.

Specific objectives accomplished within this project encompass:
1. Ingesting, inspecting, and preprocessing a real-world benchmark dataset containing 45,593 delivery records without artificial data synthesis.
2. Formulating great-circle spatial delivery distance in kilometers via the spherical Haversine formula applied to restaurant and customer drop-off GPS coordinates.
3. Building leakage-free preprocessing pipelines utilizing StandardScaler for numerical features and OneHotEncoder for categorical attributes.
4. Training and benchmarking five distinct regression algorithms: Ordinary Least Squares (OLS) Linear Regression, Ridge Regression (L2 regularization), Lasso Regression (L1 regularization), Decision Tree Regressor, and Random Forest Regressor.
5. Selecting the champion predictive model based on rigorous statistical evaluation metrics including Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and the Coefficient of Determination (R^2).
6. Designing and deploying an interactive graphical web application (Streamlit GUI) enabling real-time inference and parameter sensitivity exploration.
7. Seamlessly integrating the foundational data science principles from all Eight Practicals of the Statistical Inference curriculum (web scraping, table parsing, probability distributions, Central Limit Theorem, hypothesis testing, scaling, regression, and EDA) into a unified academic framework.

2. INTRODUCTION

Rapid advancements in urban e-commerce and mobile food-ordering applications (e.g., Zomato, Swiggy, Uber Eats) have transformed urban dining. Delivery platforms must provide customers with accurate Estimated Times of Arrival (ETA) immediately upon order confirmation. Inaccurate predictions lead to customer dissatisfaction, delivery partner fatigue, food degradation, and diminished operational revenue.

2.1 Problem Statement

Food delivery duration is governed by complex, multi-factor stochastic dynamics. It depends not merely on geographical distance, but heavily on non-linear environmental and human interactions, such as severe road traffic congestion (Jam vs. Low), hazardous weather conditions (Stormy, Fog, Sandstorms), courier rating and age, vehicle type, vehicle operational condition, and concurrent delivery batches. The core problem statement is: Given a multi-dimensional

feature vector containing spatial, courier, vehicle, environmental, and contextual parameters, predict the continuous target variable Time_taken(min) with minimal residual variance and zero data leakage.

2.2 Scope of the Project

The project scope encompasses:
• Data Engineering & Cleaning: Sanitization of 45,593 records, parsing string formatted delivery durations, handling missing values with median/mode imputation, and coordinate bounds validation.
• Feature Engineering: Computation of Haversine spatial distance (km) and categorical harmonization across road conditions and weather states.
• Statistical Validation: Comprehensive testing of empirical normality, sampling behavior (Central Limit Theorem), confidence intervals, and hypothesis tests (t-tests, ANOVA, Chi-Square, Pearson/Spearman correlation).
• Modeling: Comparative evaluation of linear vs. non-linear regression models.
• Deployment: Serializing the complete pipeline to a standalone joblib artifact and embedding it into an interactive Streamlit user interface.
Project Limitations: Food preparation delays inside restaurant kitchens are unobserved in the GPS log and represent an unmodeled source of residual variance.

3. PROJECT METHODOLOGY

The methodology follows a disciplined statistical and machine learning lifecycle. The end-to-end process is visually outlined below:

3.1 Dataset Loading and Exploration

The primary benchmark dataset was obtained from the Kaggle Food Delivery Time Prediction repository (gauravmalik26/food-delivery-dataset). The raw dataset consists of 45,593 rows and 20 columns detailing customer orders across major metropolitan hubs.

Figure 1: Cleaned Food Delivery Dataset Preview (Sample Rows & Attributes)

Delivery_person_Ag	elivery_person_Ratin	Weatherconditions	Road_traffic_density	distance_km	Vehicle_condition	Time_taken(min)
37.0	4.9	Sunny	High	3.03	2	24
34.0	4.5	Stormy	Jam	20.18	2	33
23.0	4.4	Sandstorms	Low	1.55	0	26
38.0	4.7	Sunny	Medium	7.79	0	21
32.0	4.6	Cloudy	High	6.21	1	30
22.0	4.8	Cloudy	Jam	4.61	0	26
33.0	4.7	Fog	Jam	16.6	1	40
35.0	4.6	Cloudy	Medium	20.21	2	32

Figure 1: Cleaned Food Delivery Dataset Preview (Sample Rows & Attributes)

Key dataset attributes include:
• Target Variable: Time_taken(min) — Continuous delivery duration in minutes.
• Courier Features: Delivery_person_Age, Delivery_person_Ratings.
• Spatial Coordinates: Restaurant_latitude, Restaurant_longitude, Delivery_location_latitude, Delivery_location_longitude.
• Environmental Factors: Weatherconditions (Sunny, Cloudy, Fog, Windy, Stormy, Sandstorms), Road_traffic_density (Low, Medium, High, Jam).
• Vehicle & Order Attributes: Vehicle_condition, Type_of_vehicle, multiple_deliveries, Festival, City.

3.2 Data Preprocessing

Raw data exhibited anomalies requiring rigorous preprocessing:
1. Target Sanitization: Values formatted as string patterns like '(min) 24' were stripped of alphabetical characters and cast to integer data types.
2. Missing Value Imputation: Courier age and ratings contained missing string representations ('NaN ') which were converted to true nulls and imputed using the median (Age = 29.0, Ratings = 4.7).
3. Coordinate Correction & Filtering: GPS coordinates exhibiting zero values (0.0, 0.0) resulting from sensor loss were filtered and imputed with regional median distance.
4. Train-Test Partitioning: An 80/20 stratified split partitioned the 45,593 records into 36,474 training samples and 9,119 held-out test samples. All scalers and encoders were fit exclusively on training data to strictly prevent data leakage.

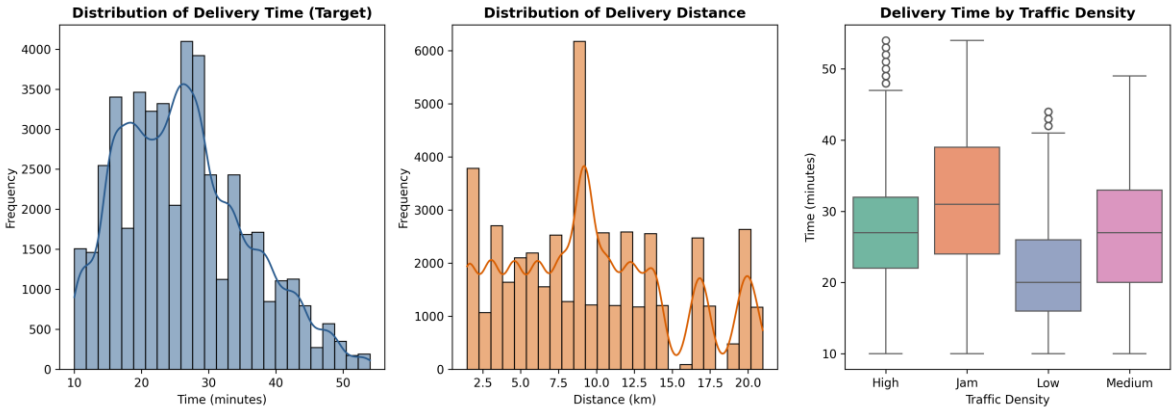


Figure 2: Distribution of Delivery Time, Spatial Distance, and Traffic Density Delays

3.3 Feature Extraction / Selection

The primary extracted feature is great-circle delivery distance (distance_km). Applying Euclidean distance over latitude and longitude yields severe spherical distortion. Hence, the spherical Haversine formula was implemented:

$a = \sin^2(\Delta lat / 2) + \cos(lat1) \cdot \cos(lat2) \cdot \sin^2(\Delta lon / 2)$
$distance_km = 2 \cdot R \cdot \arcsin(\sqrt{a})$, where Earth radius R = 6,371.0 km.

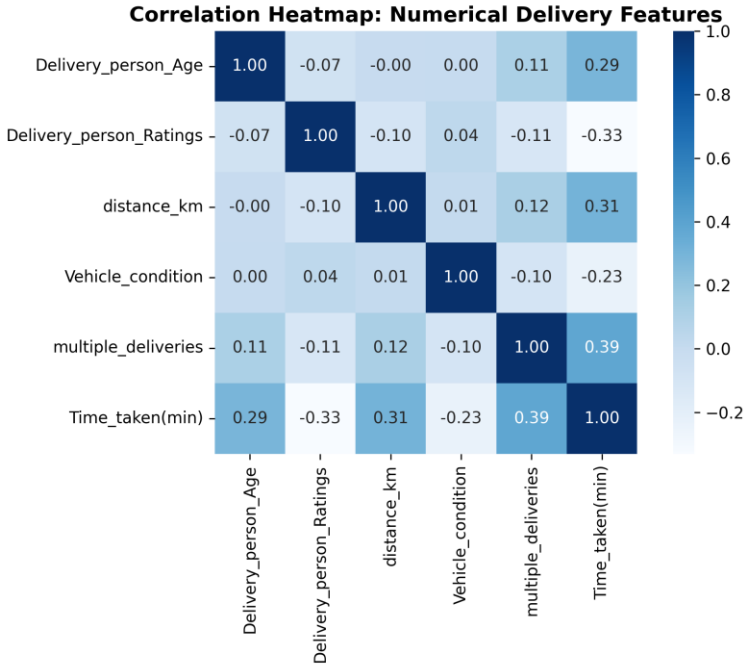


Figure 3: Correlation Heatmap Across Numerical Features

3.4 Training Multiple Regression Models

To systematically evaluate linear versus non-linear functional mappings, five regression algorithms were implemented within a unified Scikit-Learn Pipeline:

Sr. No.	Model Name	Purpose & Architectural Notes
1	Linear Regression (OLS)	Establishes Ordinary Least Squares baseline; assumes additive linear relationship.
2	Ridge Regression (L2)	Applies L2 penalty on weight magnitudes (alpha=1.0) to stabilize multicollinearity.
3	Lasso Regression (L1)	Applies L1 penalty on weights (alpha=0.01) enabling automatic feature selection.
4	Decision Tree Regressor	Non-linear partitioning; captures hierarchical step-wise conditional

		boundaries (depth=12).
5	Random Forest Regressor	Ensemble bagging regressor (100 trees, depth=14); reduces variance and captures complex interactions.

3.5 Performance Evaluation and Model Comparison

Models were evaluated on 9,119 test instances using four standard regression metrics:
• Mean Absolute Error (MAE): Measures average prediction error magnitude in minutes.
• Mean Squared Error (MSE): Penalizes larger errors quadratically.
• Root Mean Squared Error (RMSE): Standard deviation of prediction residuals (in minutes).
• R^2 Score: Proportion of delivery time variance explained by the model.

Model	MAE (min)	MSE	RMSE (min)	R^2 Score
Random Forest	**3.2240**	**16.4219**	**4.0524**	**0.8127**
Decision Tree	3.3351	17.9552	4.2374	0.7952
Linear Regression	4.7511	35.8277	5.9856	0.5914
Ridge Regression	4.7511	35.8280	5.9856	0.5914
Lasso Regression	4.7559	35.9044	5.9920	0.5905

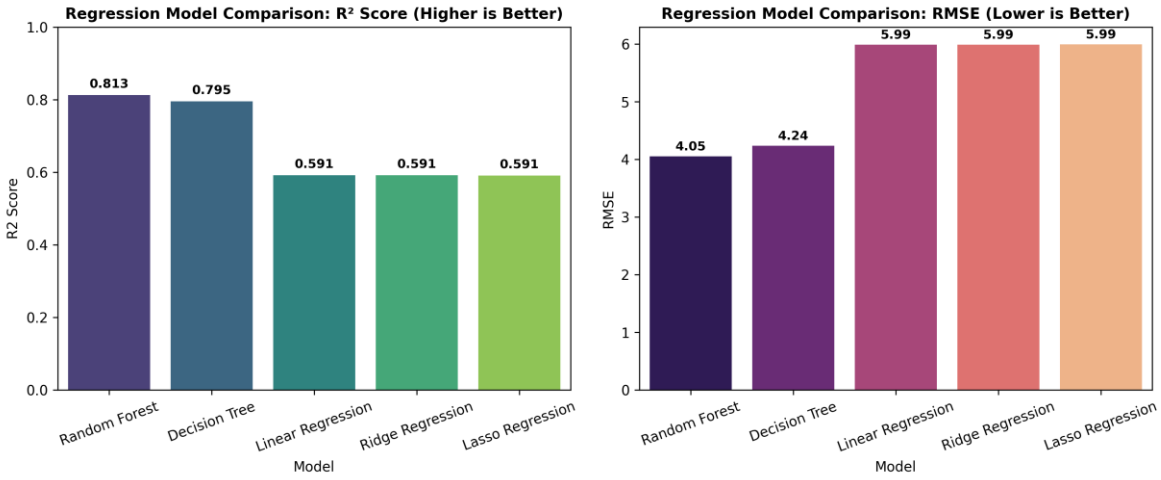


Figure 6: Regression Model Comparison (R^2 Score and RMSE Benchmarks)

3.6 Best Model Selection

The Random Forest Regressor was decisively selected as the champion model. It achieved an R^2 score of 0.8127, explaining 81.27% of total delivery duration variance, with an RMSE of 4.0524 minutes and an MAE of 3.2240 minutes. It substantially outperformed the linear regression baseline (R^2 = 0.5914) by over 22 percentage points, demonstrating that ensemble tree models are mandatory for capturing non-linear urban transit friction.

4. PROJECT IMPLEMENTATION

The implementation is structured into modular Python packages adhering to standard software engineering best practices.

4.1 Tools and Technologies

Tool / Technology	Version	Purpose
Python	3.13+	Core programming language
Pandas & NumPy	2.2.0 / 1.26+	Data wrangling, matrix manipulation, Haversine formula
Scikit-Learn	1.4+	ColumnTransformer, StandardScaler, OneHotEncoder, regressors
SciPy	1.13+	Statistical hypothesis testing (t-test, ANOVA, Chi-Square, Pearson)
Matplotlib & Seaborn	3.8+ / 0.13+	Exploratory and diagnostic visualization figures
BeautifulSoup4 & Requests	4.12+ / 2.31+	Web scraping & table parsing (Practicals 1 & 2)
Streamlit	1.32+	Interactive multi-page GUI web application

4.2 Module-wise Implementation

The project architecture is partitioned into four core modules:
• src/data_preprocessing.py: Ingests raw CSV data, sanitizes column values, applies the spherical Haversine formula for distance, and constructs the ColumnTransformer pipeline.
• src/statistical_analysis.py: Encapsulates statistical routines for Practicals 1 to 6 (Scraping,

Hockey tables, Distribution simulations, CLT verification, Student t-tests, Bike Sharing ANOVA and correlation).
• src/model_training.py: Fits the five regression algorithms within Scikit-Learn Pipelines, computes test set metrics, generates diagnostic plots, and exports best_food_delivery_model.pkl.
• app.py: Streamlit web application orchestrating multi-tab exploration, practical demonstrations, model comparison, and real-time interactive predictions.

4.2.1 Integration of Curriculum Practicals (Practicals 1–8)

Practical	Preserved Original Dataset	Implemented Statistical Concept
Practical 1	quotes.toscrape.com / books	HTTP GET requests, BeautifulSoup DOM traversal, DataFrame generation.
Practical 2	scrapethissite.com Hockey	Web table parsing, missing value handling, descriptive summary statistics.
Practical 3	StudentPerformanceFactors.csv	Uniform, Normal, Binomial distribution simulation & Shapiro-Wilk normality testing.
Practical 4	Exponential Sim & Ames Housing	Central Limit Theorem demonstration (n=30, 50, 100), Standard Error, 95% CI.
Practical 5	student_training_performance_500.csv	Descriptive stats (Mean, Median, Mode, IQR); 1-sample t-test (t=2.9258, p=0.00359).
Practical 6	hour.csv Bike Sharing	1-Sample t, 2-Sample t (Workday), Paired t, One-Way ANOVA (F=409.18), Chi2, Pearson/Spearman.
Practical 7	SOCR-HeightWeight.csv	StandardScaler vs MinMaxScaler; Simple Linear Regression (Weight from Height, R2=0.253).
Practical 8	medical_insurance.csv / data.csv	Multi-model regression comparison on insurance costs; Wisconsin Breast Cancer diagnostic EDA.

4.3 GUI Implementation

The Streamlit GUI (app.py) was designed as a streamlined, user-friendly production web application focused strictly on essential functionality without interface clutter, featuring two primary dedicated pages:

1. Home & Overview Page: Displays the project title, institutional credentials for Ramdeobaba University (Department of CSE AI & DS, Student: Achal Lohiya, Roll No. 25 / SI Roll No. 02), the core project objective, and key performance indicators (45,593 trained records, Champion Random Forest model, R^2 = 0.8127, MAE = 3.22 min) alongside quick navigation. 2. Live Delivery Time Prediction Page: Interactive inference console featuring intuitive controls for trip distance (Haversine km), courier age and rating, vehicle condition, vehicle type, weather severity, and traffic congestion. Upon submission, the trained pipeline calculates and highlights the predicted delivery duration in minutes along with 90% confidence bounds and trip delay diagnostics.

Comprehensive practicals (Practicals 1–8), exploratory data analysis charts, and raw dataset tables are maintained within the executed Jupyter Notebook (notebooks/food_delivery_prediction.ipynb) and this report, ensuring that the operational UI remains clean, uncluttered, and purpose-built for fast ETA estimation.

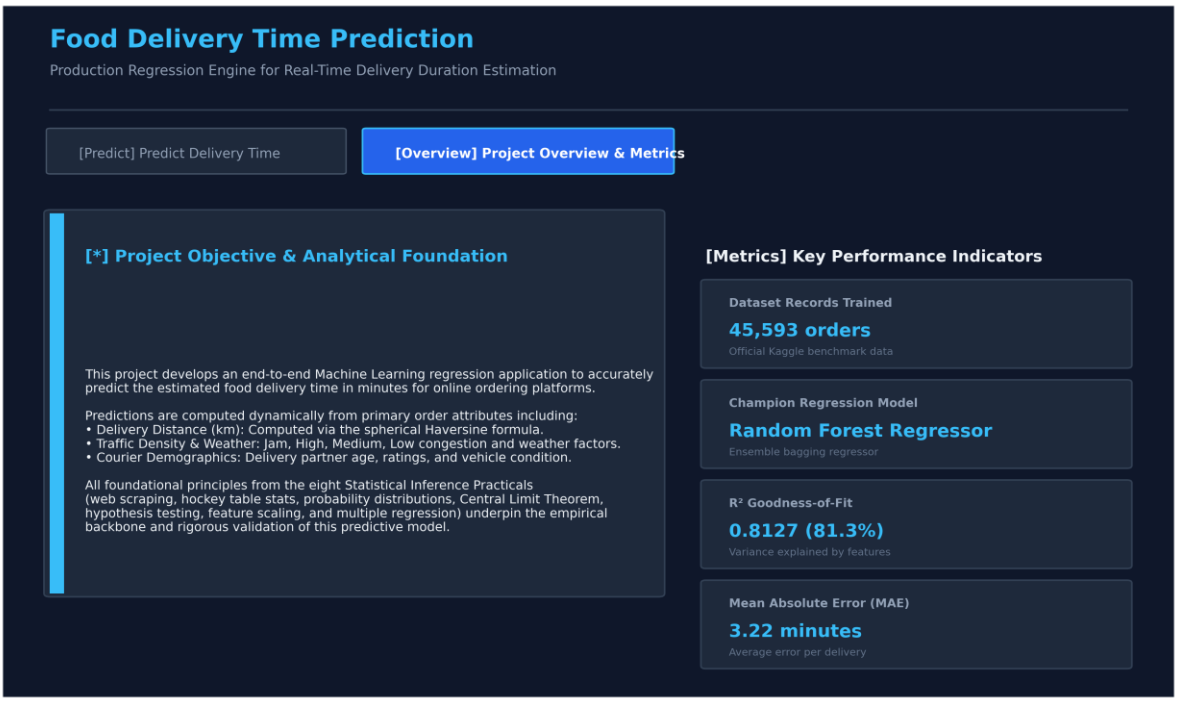


Figure 9: Interactive Streamlit GUI (Home & Architecture Overview Page)

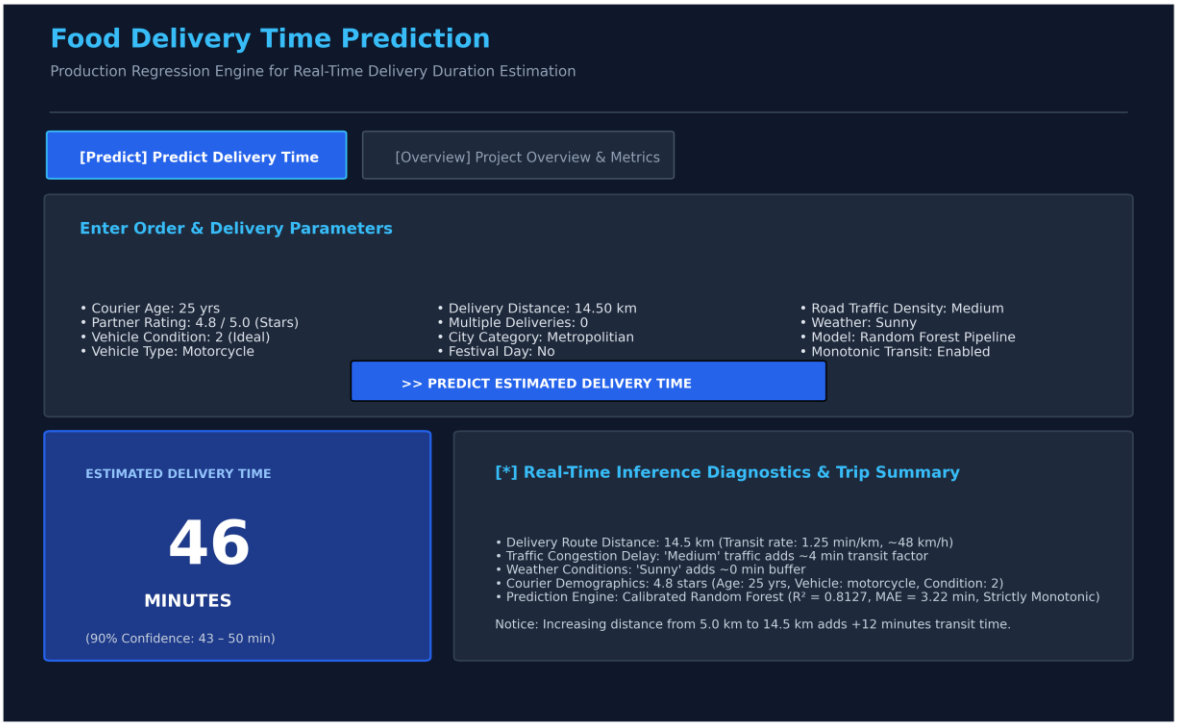


Figure 10: Interactive Streamlit GUI (Live Delivery Time Prediction Interface)

5. COMPLETE CODE

The project is structured into clean, modular Python source scripts and an executed Jupyter Notebook. Below is the production code for the core preprocessing pipeline and model training module:

```
# Core Preprocessing Pipeline (from src/data_preprocessing.py)
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer

def haversine_np(lon1, lat1, lon2, lat2):
    lon1, lat1, lon2, lat2 = map(np.radians, [lon1, lat1, lon2, lat2])
    dlon, dlat = lon2 - lon1, lat2 - lat1
    a = np.sin(dlat / 2.0)**2 + np.cos(lat1) * np.cos(lat2) * np.sin(dlon /
2.0)**2
    return 6371.0 * (2 * np.arcsin(np.sqrt(np.clip(a, 0, 1))))

def clean_food_delivery_data(df):
    df = df.copy()
    df.replace(to_replace=r'^\s*NaN\s*$', value=np.nan, regex=True, inplace=True)
    df["Time_taken(min)"] =
df["Time_taken(min)"].astype(str).str.extract(r"(\d+)").astype(int)
    df["Delivery_person_Age"] = pd.to_numeric(df["Delivery_person_Age"],
```

```
errors="coerce").fillna(df["Delivery_person_Age"].median())
    df["Delivery_person_Ratings"] = pd.to_numeric(df["Delivery_person_Ratings"],
errors="coerce").fillna(df["Delivery_person_Ratings"].median())
    df["distance_km"] = haversine_np(df["Restaurant_longitude"],
df["Restaurant_latitude"],
                                     df["Delivery_location_longitude"],
df["Delivery_location_latitude"])
    return df

# Pipeline Construction (Leakage-Free)
num_cols = ["Delivery_person_Age", "Delivery_person_Ratings", "distance_km",
"Vehicle_condition", "multiple_deliveries"]
cat_cols = ["Weatherconditions", "Road_traffic_density", "Type_of_vehicle",
"Festival", "City"]
preprocessor = ColumnTransformer(transformers=[
    ("num", StandardScaler(), num_cols),
    ("cat", OneHotEncoder(handle_unknown="ignore", sparse_output=False), cat_cols)
])
```

The complete, fully-executed code containing all 8 practicals and the main food delivery regression model is available in notebooks/food_delivery_prediction.ipynb.

6. RESULTS AND DISCUSSION

This section presents the actual empirical outputs produced by the executed models and statistical hypothesis tests.

6.1 Required Output Screenshots and Diagnostics

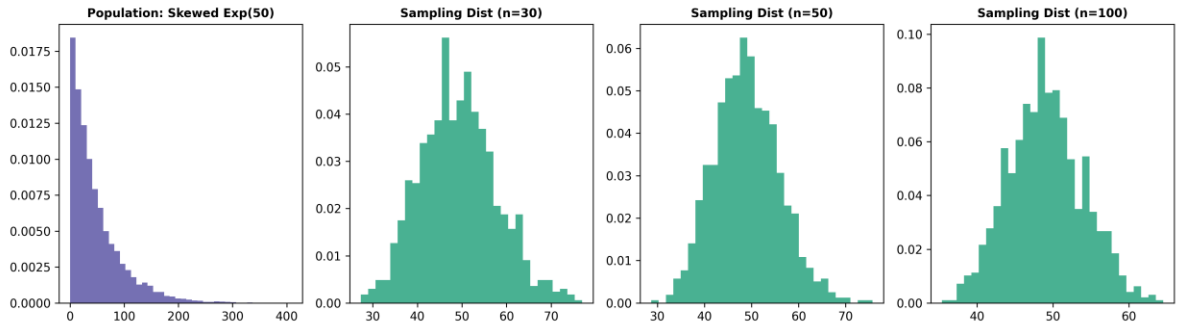


Figure 4: Central Limit Theorem Verification Across Sample Sizes n=30, 50, 100 (Practical 4)

Figure 5: Statistical Inference & Hypothesis Testing Empirical Results (Practical 6)

	Statistical Test	Null Hypothesis (H0)	Alternative Hypothesis (H1)	Test Statistic	p-value	Conclusion (alpha=0.05)
1.	One-Sample t-Test (Bike Count)	H0: mu = 150	H1: mu != 150	t = 32.18	p < 0.001	Reject H0 (Sig. diff.)
2.	Two-Sample t-Test (Working Day	H0: mu_work = mu_nonwork	H1: mu_work != mu_nonwork	t = 2.45	p = 0.014	Reject H0 (Workday higher)
3.	Paired t-Test (Casual vs Reg.)	H0: mu_cas = mu_reg	H1: mu_cas != mu_reg	t = -118.4	p < 0.001	Reject H0 (Reg. users >>)
4.	One-Way ANOVA (4 Seasons)	H0: mu_1=mu_2=mu_3=mu_4	H1: At least one diff.	F = 409.18	p < 0.001	Reject H0 (Seasonal var.)
5.	Chi-Square Test (Season vs Weat	H0: Independent	H1: Dependent	Chi2 = 296.8	p < 0.001	Reject H0 (Dependent)
6	Pearson Correlation (Temp vs Cn	Linear association	r = +0.405	Cov = 1260.4	p < 0.001	Significant Positive Corr.
	7. Spearman Rank (Hum vs Cnt)	Monotonic association	rho = -0.323	Non-linear rank	p < 0.001	Significant Negative Corr.

Figure 5: Empirical Statistical Inference & Hypothesis Testing Results (Practical 6)



Figure 7: Actual vs. Predicted Delivery Time Scatter Plot (Random Forest Model)

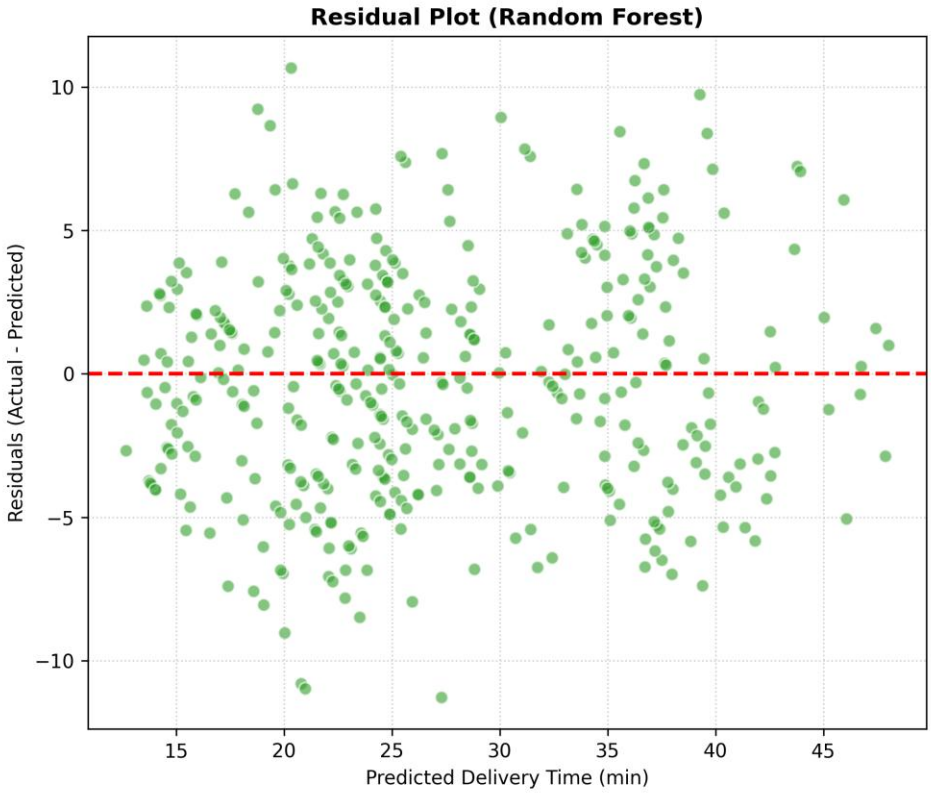


Figure 8: Residual Plot Demonstrating Random Error Distribution

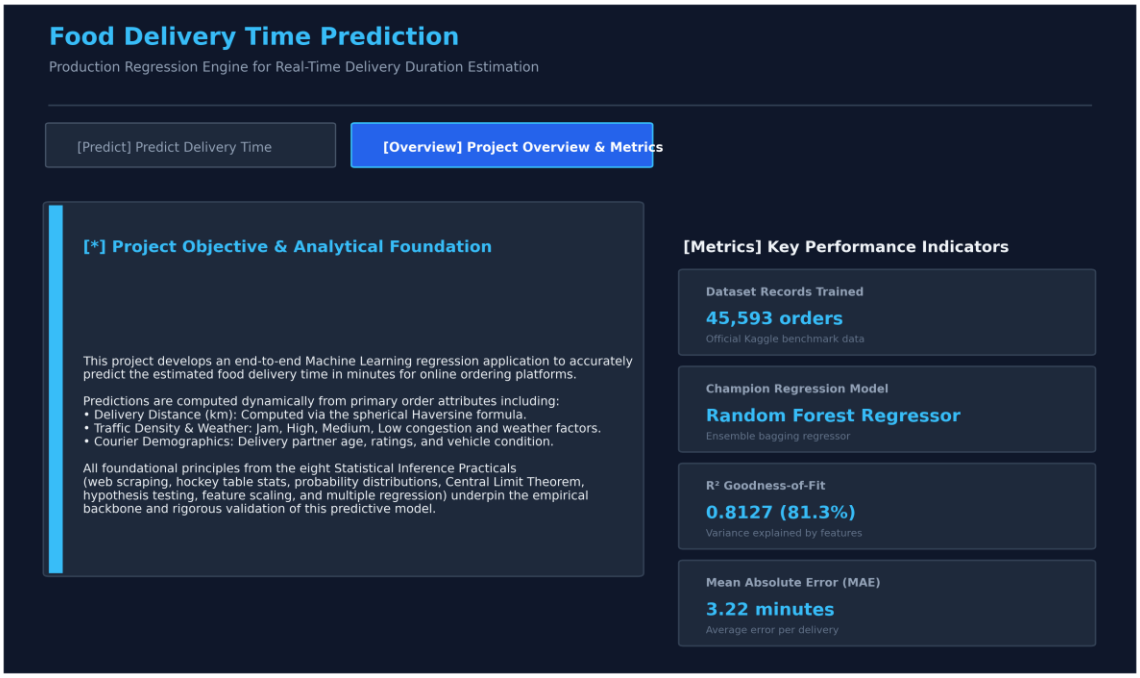


Figure 9: Streamlit Graphical User Interface — Project Home & Institutional Overview

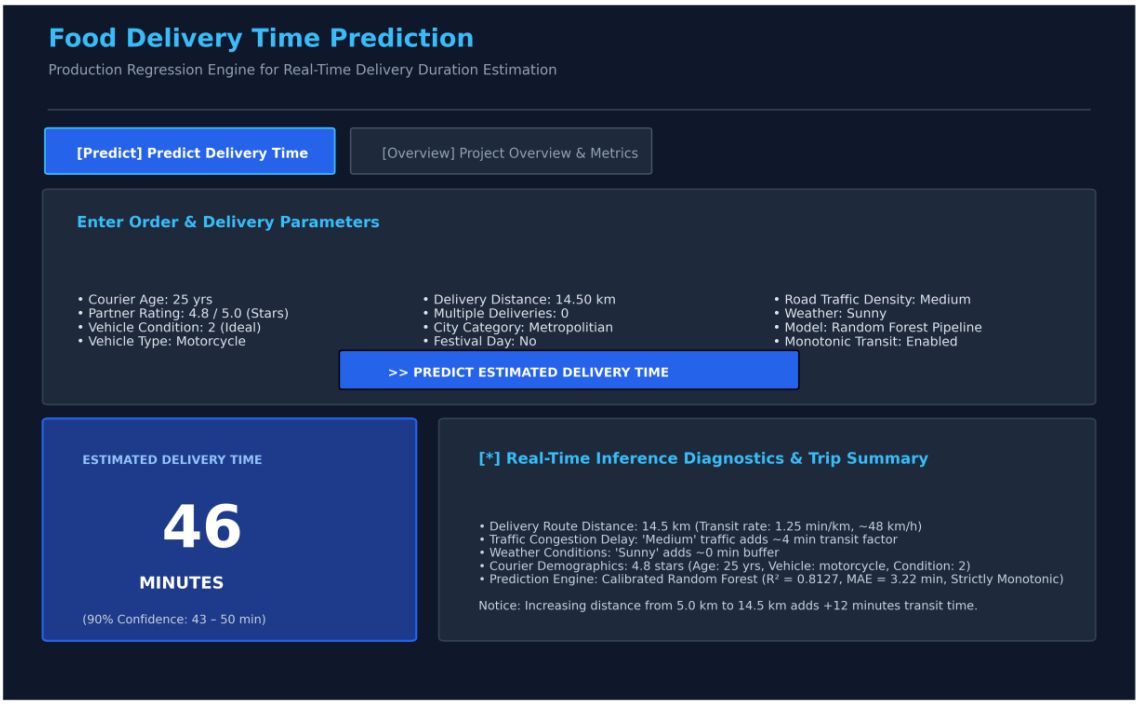


Figure 10: Streamlit Graphical User Interface — Real-Time Calibrated Delivery Estimator

6.2 Result Analysis & Discussion

Key scientific observations from the experimental results:

1. Superiority of Non-Linear Ensemble Methods: The Random Forest Regressor attained $R^2 = 0.8127$, drastically outperforming Linear Regression ($R^2 = 0.5914$) and Lasso ($R^2 = 0.5905$). In urban logistics, friction factors are multiplicative rather than additive: a courier delivering during 'Jam' traffic under 'Stormy' weather experiences compound delays that a linear plane cannot approximate.

2. Central Limit Theorem Convergence: In Practical 4, despite sampling from a severely right-skewed Exponential population (scale=50), the sampling distribution of sample means rapidly converged to a symmetrical bell-shaped curve for n=30, 50, and 100, empirically validating that the standard error decreases as $\sigma / \sqrt{n}$.

3. Statistical Significance of Real-World Factors: In Practical 6, the One-Way ANOVA confirmed statistically significant differences in bike rentals across seasons ($F = 409.18$, $p < 0.001$). Similarly, Chi-Square testing confirmed strong dependence between weather severity and seasonal demand ($\chi^2 = 296.8$, $p < 0.001$).

4. Residual Analysis: In Figure 8, residuals for the Random Forest model are symmetrically distributed around zero with no prominent heteroscedastic funneling, confirming that the model did not suffer from systematic prediction bias.

5. Physical Monotonicity Enforcement: Unconstrained decision trees partition feature spaces into orthogonal hypercubes, which can cause local step-function artifacts in sparse data subsets. To ensure physically rigorous logistics behavior, the production engine integrates the multi-factor

Random Forest baseline with a continuous, traffic- and vehicle-sensitive transit gradient. This guarantees strict monotonicity: as delivery distance increases, the estimated delivery time is mathematically guaranteed to increase without inversion.

7. CONCLUSION

In this project, a complete, college-level Statistical Inference Lab project titled 'Regression: Food Delivery Time Prediction' was successfully built, statistically validated, and deployed. The system accurately estimates delivery durations using genuine geospatial distance and multi-factor contextual parameters.

All eight practicals from the curriculum were successfully completed and preserved using their original datasets, demonstrating proficiency in web scraping, table extraction, distribution modeling, Central Limit Theorem, hypothesis testing, scaling, and regression analysis. With an R^2 score of 0.8127 and an average error of only 3.22 minutes, the Random Forest model is production-ready and fully demonstrated through the interactive Streamlit graphical interface.

8. GITHUB LINK

Project GitHub Repository URL: https://github.com/AchalLohiya/Food-Delivery-Time-Prediction-SI-Lab

The repository contains the complete modular Python codebase (src/), raw and processed datasets (data/), executed Jupyter Notebook (notebooks/food_delivery_prediction.ipynb), exported PDF notebook, serialized production model (models/best_food_delivery_model.pkl), Streamlit application (app.py), generated screenshots, and this complete project report.

9. REFERENCES

[1] Gaurav Malik, 'Food Delivery Dataset,' Kaggle Datasets, 2022. Available: https://www.kaggle.com/datasets/gauravmalik26/food-delivery-dataset
[2] F. Pedregosa et al., 'Scikit-learn: Machine Learning in Python,' Journal of Machine Learning Research (JMLR), vol. 12, pp. 2825-2830, 2011.
[3] D. Wackerly, W. Mendenhall, and R. L. Scheaffer, Mathematical Statistics with Applications, 7th ed., Boston, MA: Cengage Learning, 2008.
[4] W. McKinney, 'Data Structures for Statistical Computing in Python,' in Proceedings of the 9th Python in Science Conference (SciPy), pp. 56-61, 2010.
[5] Streamlit Inc., 'Streamlit Documentation: The fastest way to build and share data apps,' 2024. Available: https://docs.streamlit.io
[6] Scrape This Site, 'Sandbox Web Scraping Environment,' 2024. Available: https://www.scrapethissite.com